

1. Document Structure & Chunking Logic

The source document (AI Training Document.pdf) is processed through a multi-stage pipeline to prepare it for retrieval-augmented generation.

1.1 Text Extraction

PyMuPDF (fitz) extracts raw text from each page of the PDF document. This library provides reliable text extraction while preserving the logical reading order. Each page's text is stored with its page number for source attribution.

1.2 Text Cleaning

The extracted text undergoes several cleaning steps:

- Removal of standalone page numbers
- Fixing of hyphenated words broken across lines
- Normalization of excessive whitespace and blank lines
- Stripping of leading/trailing whitespace from each line

1.3 Chunking Strategy

Documents are split into 100-300 word segments using LangChain's RecursiveCharacterTextSplitter with the following configuration:

- Chunk size: 800 characters (~200 words)
- Chunk overlap: 150 characters (ensures context continuity)
- Separators: paragraph breaks, line breaks, sentence endings, commas, spaces
- Minimum chunk size: 10 words (filters out fragments)

The sentence-aware splitting ensures chunks break at natural boundaries, preserving semantic coherence within each segment.

2. Embedding Model & Vector Database

2.1 Embedding Model: all-MiniLM-L6-v2

We use the all-MiniLM-L6-v2 model from sentence-transformers for generating 384-dimensional dense vector embeddings. This model was chosen for:

- Compact size (22M parameters) enabling fast inference
- Strong performance on semantic textual similarity (STS) benchmarks
- Wide adoption in production RAG systems
- Sub-second embedding generation for real-time applications

2.2 Vector Database: ChromaDB

ChromaDB serves as the persistent vector store with the following configuration:

- Storage: Persistent client (data survives across sessions)
- Distance metric: Cosine similarity (HNSW index)
- Metadata: Each chunk stores source page number, word count, and chunk ID

- Batch indexing: Documents are indexed in batches of 50 for memory efficiency

3. Prompt Format & Generation Logic

3.1 LLM: Mistral 7B via Ollama

The Mistral 7B Instruct model is used for response generation, served locally through Ollama. Key advantages include:

- Instruction-tuned for accurate prompt following
- Efficient local inference without API costs
- Native streaming support for real-time responses
- Strong performance on RAG-specific tasks

3.2 Prompt Template

The prompt follows a structured format:

1. SYSTEM PROMPT: Instructs the LLM to answer only from provided context, cite sources, and acknowledge when information is insufficient.
2. USER PROMPT: Combines retrieved context passages (labeled with source numbers and page references) with the user's question.

This design minimizes hallucination by grounding all responses in the retrieved documents and encouraging explicit source citation.

3.3 Streaming Implementation

Responses are streamed token-by-token using Ollama's streaming API. The Streamlit frontend uses `st.write_stream()` to render tokens as they arrive, providing a responsive user experience. Source passages are displayed in an expandable section below each response.

4. Example Queries & Responses

Below are representative queries tested against the system. Results demonstrate both successful retrieval and edge cases.

Example 1: "What is this document about?"

SUCCESS - The system correctly identifies the document's topic and provides a comprehensive overview citing multiple source passages.

Example 2: "What are the main topics covered?"

SUCCESS - Retrieves relevant chunks from different sections and synthesizes a structured list of topics.

Example 3: "What guidelines or rules are mentioned?"

SUCCESS - Accurately extracts specific guidelines with proper source attribution.

Example 4: "What is quantum computing?"

EXPECTED FAILURE - The system correctly responds that the provided documents do not contain information about quantum computing, demonstrating grounding.

Example 5: "Summarize the document in one paragraph."

PARTIAL SUCCESS - Provides a good summary but may miss some sections if they are not captured in the top-k retrieved chunks.

5. Limitations & Observations

- Hallucination: While grounding prompts significantly reduce fabrication, the model may occasionally paraphrase loosely from context.
- Context Window: With top-5 retrieval, very long chunks may exceed the effective context window, leading to truncated attention.
- Latency: First query after startup is slower due to model loading. Subsequent queries benefit from cached model weights.
- PDF Quality: Text extraction depends on PDF formatting. Scanned documents, tables, and images are not processed.
- Single Document: The current implementation indexes a single document. Multi-document support would require metadata-based filtering.